

GABARITO

Considere a definição abaixo, da estrutura do nó de uma árvore binária:

```
typedef char tDado;
struct regNo
{
    struct regNo * esq;
    tDado dado;
    struct regNo * dir;
};
typedef struct regNo No;
typedef No * PtNo;
```

1) Construa uma função que receba um ponteiro (não nulo) para a raiz de uma árvore, e um valor *v* do tipo *char*, e realize a inserção de um novo nó (armazenando *v*) à esquerda do nó que contenha um valor *x*. Deve ser retornado um código para indicar a (im)possibilidade (0 ou 1, true ou false).

Criação de novo nó em C e em C++, respectivamente: (3 pts)

PtNo q = (No*) malloc(sizeof(No));

PtNo q = new No;

```
ptNo localizaNo(ptNo p, tDado v)
{
    ptNo buscaEsq;

    if (p == NULL)    // não há elementos a procurar
        return NULL;
    if (p->dado == v)  // elemento encontrado
        return p;
    // tentativa subárvore da esquerda - se a busca teve sucesso, já se retorna
    buscaEsq = localizaNo(p->esq, v);
    if (buscaEsq != NULL)
        return buscaEsq;
    // tentativa na subárvore da direita - é retornado, com sucesso ou não
    return localizaNo(p->dir, v);
}
```

```
bool insereEsquerda(ptNo p, tDado x, tDado v)
{
    ptNo q, r;

    q = localizaNo(p, x);
    if (q == NULL)    // não foi encontrado o valor x
        return false;
    if (q->esq != NULL) // já existe filho à esquerda
        return false;
    r = new No;
    r->dado = v;
    r->esq = NULL;
    r->dir = NULL;
    q->esq = r;
}
```

ATENÇÃO: é problemático fazer os dois testes de uma só vez como abaixo:

```
if (q == NULL || q->esq != NULL)    // não foi encontrado o valor ou já
    return false;                    // existe filho à esquerda
```

...pois se *q* guarda NULL, "*q->esq*" não é uma expressão válida

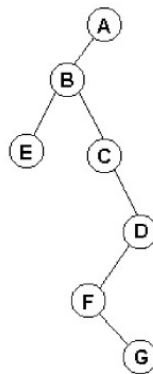
Pode ser que a linguagem não faça o segundo teste se o primeiro falhar,

mas como isso não é garantido em qualquer linguagem a abordagem mais técnica (mais segura) é separar os testes.

2) Construa uma função que receba um ponteiro para a raiz de uma árvore de inteiros (tipo int) e retorne a soma dos elementos de toda a árvore. (2 pts)

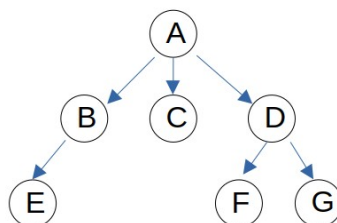
```
bool somaElementos(ptNo p)
{
    if (p == NUL)
        return 0;
    return p->dado + somaElementos(p->esq) + somaElementos(p->dir);
}
```

3) Considere a árvore binária abaixo. Com seria a sua impressão, se fosse feito um percurso: a) em pré-ordem; b) em ordem central (ou simétrica); em ordem final (pós-ordem) (1,5 pt)



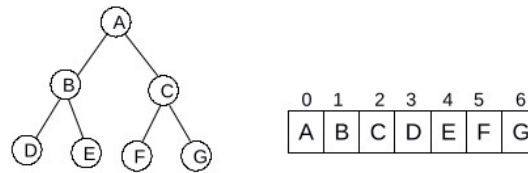
Pré ordem: A B E C D F G
Ordem central: E B C F G D A
Pós ordem: E G F D C B A

4) Se considerarmos que a árvore acima é uma árvore transformada em árvore binária, qual a árvore original? (1,5 pt)



5) Se considerarmos a possibilidade de armazenar uma árvore binária em um array, como seria a lógica de armazenamento e interpretação da hierarquia? Quais as vantagens e desvantagens de empregar tal solução (armazenamento em array). Exemplifique. (2 pts)

A solução comum seria como ilustrado abaixo para 7 nós



A partir da posição i de um nó:

O pai daquele nó está em $(i-1) \text{ div } 2$, para $i > 0$. (sendo “div” a divisão inteira)
(se $i=0$, trata-se da raiz)

O filho à esquerda do nó está na posição $2i+1$, se $2i+1 \leq n-1$
(se $2i+1 > n-1$, o nó não tem filho à esquerda).

O filho à direita do nó está na posição $2i+2$, se $2i+2 \leq n$
(se $2i+2 > n-1$, o nó não tem filho à direita).

Como vantagem da solução, somente se emprega a memória necessária (não há custo de memória com ponteiros e outros dados para garantir o conhecimento da hierarquia).

Como desvantagem, o array pode oferecer limites ao crescimento da árvore, sendo necessário superestimar o tamanho do array para evitar esse problema.